

YG IAspire 2.0

Title : XONE Bot – A Conversational Text-to-SQL Agent

Theme: Future Of Banking: GenAI, Innovation & Human-AI Collaboration

Team Members: *Ganta Srilalitha (Empld: 31590)*
 Rithik Gundeti

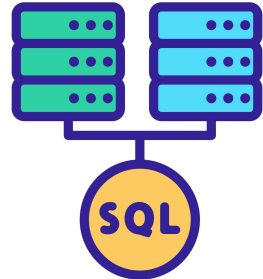
Problem/Opportunity

a. What problem are we solving?

- Regulatory analysts worked with **multiple complex database tables** having **large schemas**.
- Retrieving data required **manual SQL writing** involving joins, filters, aggregations, and group-by clauses.
- SQL queries often needed **Business Analyst (BA) validation** to ensure correctness.
- Although **GitHub Copilot helped generate SQL**, users still had to **provide table schema and context**, making the process time-consuming and dependent on technical knowledge.
- **Result: Longer turnaround time, and more man-power.**

b. Who is impacted?

- **Business Unit:** XONE Regulatory (Reg)
- Business Analysts (BA)
- Data Engineers
- Operations teams using regulatory reporting data



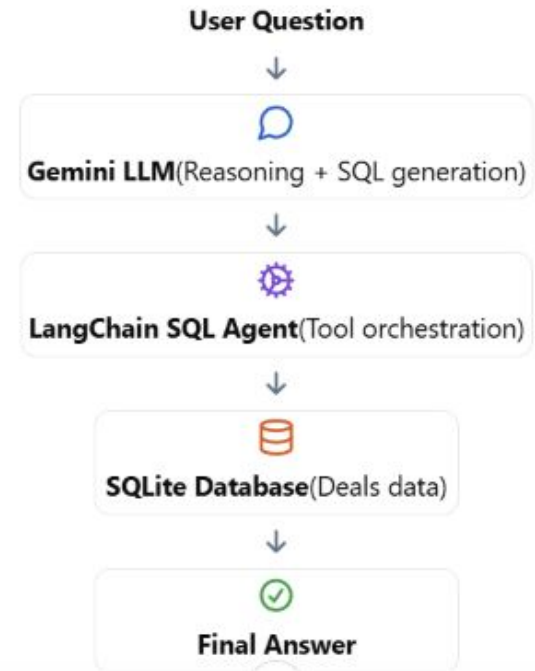
Proposed Solution

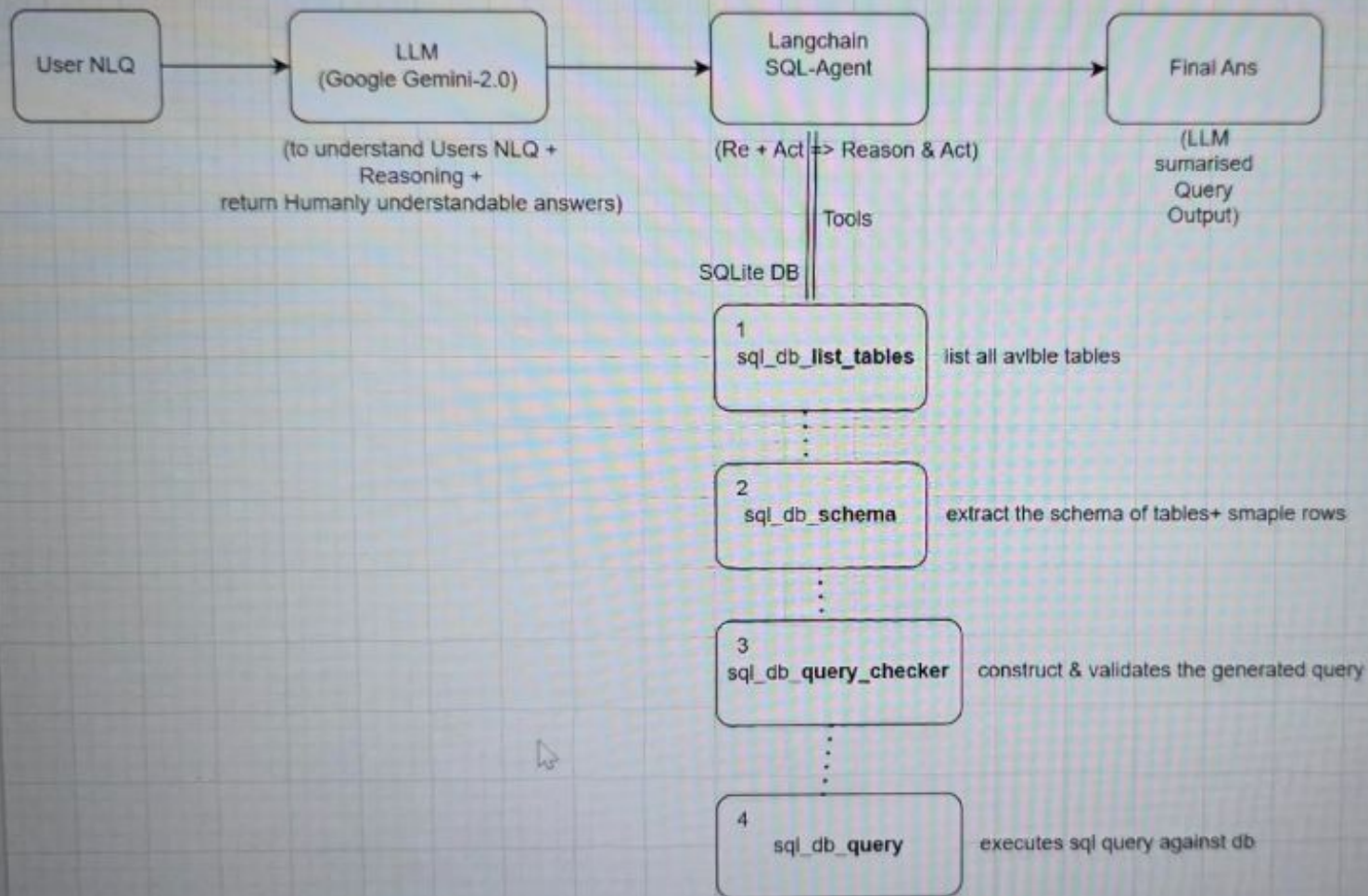
a. Idea Overview

- Develop an **LLM-powered SQL AI Agent (Text-to-SQL Agent)**.
- Users ask questions in **plain English** instead of writing SQL.
- **LLM** automatically understands the request, generates SQL
- The **agent** validates and **executes the SQL query generated**
- **Returns the results** in an easy-to-understand format.

b. Key Features / Components

- Natural Language to SQL conversion using **Google Gemini Flash 2.0 LLM**
- **LangChain SQL Agent** for intelligent **query orchestration**
- **SQLite database** connected through LangChain
- Features:
 - a. **Automatic schema understanding** and **SQL validation**
 - b. **Conversational interface** with human-readable responses
 - c. **User friendly** for tech & non-tech





Sample UseCase:

Click below for
complete
implementation. 

[Complete
Implementation](#)

```
#user_input_fetch = "Fetch codapp and numtra of deals having law as HKMA."  
user_input_fetch = "Fetch deals deals having law as HKMA."  
agent_executor.run(user_input_fetch)
```



```
> Entering new SQL Agent Executor chain...  
Action: sql_db_list_tables  
Action Input: deals I should query the schema of the `deals` table to see what columns I can query.  
Action: sql_db_schema  
Action Input: deals  
CREATE TABLE deals (  
    codapp TEXT,  
    numtra BIGINT,  
    uni TEXT,  
    law TEXT,  
    numver BIGINT,  
    redid TEXT,  
    assetclass TEXT  
)  
  
/*  
3 rows from deals table:  
codapp  numtra  uni      law      numver  redid  assetclass  
IRD      1234    IRDONE00004321  HKMA      1      xxxx123-yyy456  interest-rate  
IRD      5678    IRDONE000058765  CANADA    2      xxxx375-yyy768  interest-rate  
IRD      7642    IRDONE000062543  EMIR      3      xxxx764-yyy1765  interest-rate  
*/I should query the `deals` table for deals where the `Law` is `HKMA`.  
Action: sql_db_query_checker  
Action Input: SELECT codapp, numtra, uni, law, numver, redid, assetclass FROM deals WHERE Law = 'HKMA'  
SELECT codapp, numtra, uni, law, numver, redid, assetclass FROM deals WHERE Law = 'HKMA' LIMIT 10  
~~~~~tool_code~~~ Looks correct.  
Action: sql_db_query  
Action Input: SELECT codapp, numtra, uni, law, numver, redid, assetclass FROM deals WHERE Law = 'HKMA'  
  
> Finished chain.  
'The deals with law HKMA are: IRD, 1234, IRDONE00004321, HKMA, 1, xxxx123-yyy456, interest-rate; IRD,  
87623, IRDONE00004127364, HKMA, 2, xxxx52134-yyy76253, interest-rate; IRS, 71265, IRDONE0000982173, HK  
MA, 1, xxxx6123-yyy76235, interest-rate.'
```

Expected Benefits

a. Business Benefits

- **Faster access** to regulatory data
- **Reduced dependency on SQL experts** and Business Analysts
- Improved productivity and **reduce** longer turnaround times
- Faster report generation and **decision making**
- **Better user experience** through conversational search



b. Qualitative / Quantitative Impact

Qualitative

- Fast and Improved self-service analytics
- Reduced manual effort
- Better and Consistent SQL generation

Quantitative (*Rough Estimates*)

- **50–70% reduction** in manual SQL writing effort
- **30–50% faster** data retrieval for common queries
- Reduced BA validation effort for repetitive requests

Feasibility & Risk

a. Effort

- **Medium for POC**
- POC can be developed within **2–3 weeks**
- **Usage of Open Source LangChain components**

b. Dependencies

- Access to database/schema
- Gemini access
- LangChain framework
- Secure database connectivity

c. Risks

Risk	Mitigation
Incorrect SQL generation	SQL validation + human review
Complex business logic	Fine-tune prompts and add examples
Sensitive data access	Role-based access and security controls
AI hallucination	Execute only validated SQL against database

Recommendation & Next Steps

a. Pilot Scope

- Implement for a **limited set of regulatory tables**
- Support top **20–30 frequently used business queries**
- Validate with Regulatory Analysts and BAs

b. Immediate Actions

- Identify pilot users
- **Integrate AI assistant with existing database**
- Capture feedback and **improve prompts**
- Measure accuracy and response time

c. Decision Required

- Approval to execute a **Proof of Concept (POC)**
- Access to **sample regulatory datasets**
- Nominate pilot users from XONE Regulatory team

